1. Running for rare queries such as “larchfield audit” returned strong rankings as expected. However, one problem I encountered several times was adding plurality to search terms. For example, running the query for “larchfield audits” came back with different results. Since our logic lacks linguistic robustness, it’s also not able to distinguish the different kinds of “fire” in the query “who was fired”.
2. I decided on a tool call format `<tool_call>{"name":"search_docs", "args":{...}}</tool_call>`. I chose JSON over XML to keep the types for argument parameters. I chose tags surrounding the JSON since it was easier to read exactly when the call ends (as opposed to a one-line code fence for example). In the decoding logic, I used `raw_decode` vs regex to leverage `JSONDecoder`’s built-in functionality of knowing when the JSON terminates. Throughout the homework, I noticed the model might drift with how it invokes tools. For example, there were instances where it used `<|tool_call|>` instead - in this case I propagate the error description back to the model. Another case is when the model outputs only the JSON, which my code assumes is a tool call. In production environments, I would have left that out since users could have ask for a pure JSON reply.
3. My agent will mark reflective thoughts such as “I’ll look that up” without an actual `<tool_call>` as a final answer. Since this is V1, I did not try to over-complicated the logic by guessing the intent from such responses. Instead, I added the following rules to the system prompt:

agent_session.py

7. Do not reply with what you are going to do next (e.g. "I'll look up Cardinal Energy's reporting instructions"); do it with a tool call instead.

tool_protocol.py

If you need to call a tool, call it in the same reply and do not announce an action without calling a tool.

This is not a definitive fix but a tradeoff of simplicity vs remaining risk the model continues to misbehave.

4. I initially indexed whole documents but that blew up the context window so I index chunks of 3 non-blank lines that do not cross a section heading. The read window had a more complex thought process since it was bounded by `SNIPPET_MAX_TOKENS` and `WINDOW_MAX_TOKENS`. The snippet would start at the “best” line instead of the first line so the model had the “best” content it needed verbatim. The best line was defined as the chunk’s line with the most query word hits, and runs to the end of the chunk while respecting `SNIPPET_MAX_TOKENS`. The read window from `read_doc()` lines until the line that would push it over the `WINDOW_MAX_TOKENS` limit. These settings were enough to pass the live tests.
5. If the *stdout* and *stderr* outputs are over max_chars, I returned the beginning and end while saving the entire output into a file that the model can look up via *grep, head, tail, or sed*. I kept the head and tail since that’s usually where important information is contained (e.g. logs) and the middle is usually noise. I put the note on incomplete output first so the model knows immediately the the following output had been cut. I also treated each

stream (*stdout* vs *stderr*) differently - so if stdout was over the limit it doesn't push out the output of *stderr*. Weakness here is that core information in middle might be cut and require a second lookup.

6. During building the compaction logic, my system prompt was 1,309 tokens with a test budget of 4,000. Before every model call, if the messages totaled more than the context budget, we will drop the oldest messages after the system prompt until we are under 50% of the budget. The dropped messages gets summarized via the *aux_lm* along with any previous summaries, keeping exact numbers, names, dates, user requests/preferences, etc. The summary goes in the messages right after the system prompt, replacing the old one if exist.I decided to compact to 50% so we'd have less frequent compactions (since each costs an LM summary which adds up costs and latency).
7. I store facts in a simple append-only *jsonl* file (e.g. *memory_dir/facts.jsonl*). I chose this over a json array file since we don't need to inflate / re-serialize the object. Each line is a json object with the "fact" (e.g "*The user is on the Customer Success team and is based in Rotterdam.")* and "stored_at" fields. All stored facts go into system prompt at session start. For V1 and a handful of facts, returning every fact and ignoring the query in `recall()` worked well for the test cases. The prompt will tell the model to call *remember* when the user states a fact or preference, an optimization in later versions can be to rank or compact facts via *aux_lm*. What's missing is clarifying answers from *ask_user* might not be stored as it is considered a tool output.
8. All tools run without approval when in confirm mode, except *run_terminal*. Those tools are read-only so they cannot mutate anything outside the agent (e.g. search/read emails), although it is possible that reading a document with prompt injection will still be detrimental especially without guardrails (which we put into the prompt). The design decision to ask for every tool would hurt UX / annoy the user. *Run_terminal* needs approval because it's the only tool that can mutate the external environment (e.g. write/delete files).
9. On my Mac, *sandbox.py* stops a command from running programs not on allowlist or are given as a path (which could be a binary). The commands run in a restricted environment (OS-specific jail) without my process' environment variables (e.g. API keys), no network, no writes outside the workspace, no reads of home dir or this repo. Commands can still start other programs through the base interpreter (e.g. python3 or Python files via *find -exec*), read files elsewhere on the machine, delete/modify files in workspace, and use unlimited computer resources until timeout.
10. I can summarize the limitations of my harness in two groups: over-delegating deterministic responsibilities to the prompt and under-engineering. Regarding group 1, there were several instances where we depending on the prompt too much, the most glaring was how I guarded against prompt injection by adding a rule to the system prompt to not take instructions from data. However, for something this important, there were no checks in the code or stronger enforcement due to workarounds (e.g. the injection could come elsewhere besides the email or docs). Regarding under-engineering - this occurred in many parts of the harness. For example, the memory could only grow in size and no facts were ever evicted or evaluated.